53. Capstone: Production-Shaped HTTP API

53. Capstone: Production-Shaped HTTP API

Phase: Phase 8 — Capstones
Estimated time: 3–5 days
Slug: `53-capstone-infra`

Learning objectives

- Ship an API with graceful shutdown
- Add metrics, retries, worker pool
- Write integration tests
- Document runbook notes

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Mission

Build a small but production-shaped service:

- REST endpoints for a job queue resource
- Enqueue is fast; workers process asynchronously
- Graceful shutdown drains workers
- Structured logs + basic metrics counters
- Outbound calls use retries + timeouts
- `GET /healthz` and `GET /readyz`

Milestones

1. Skeleton server + config
2. In-memory store + handlers
3. Worker pool + backpressure
4. Observability + shutdown
5. Tests + README

Rubric (excellent)

- No goroutine leaks on shutdown
- Race-clean tests
- Clear package boundaries
- Explicit error types for API
- Timeouts everywhere

Common mistakes (especially from Python instincts)

- God-object main.go

- In-memory only without interfaces (hard to test)
- Skipping readiness vs liveness

Exercises

Work in `exercises/53-capstone-infra/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/53-capstone-infra/` until you have an honest failing attempt.

Self-check

1. What does readyz mean?
2. How prove no leaks?
3. Where put business logic?

Further reading

- https://go.dev/doc/tutorial/web-service-gin